Docker : Conteneurisation et Déploiement d'Applications

Introduction

Objectif du projet : Découvrir Docker et maîtriser la conteneurisation pour simplifier le déploiement d'applications.

Pourquoi Docker ?

- **Portabilité** : Un conteneur fonctionne partout (dev, test, prod).
- **Isolation** : Pas de conflits entre applications.
- **Efficacité** : Moins gourmand qu'une VM (partage du noyau OS).

Concepts Clés de Docker

Terme	Définition
Image	Modèle read-only pour créer un conteneur (ex: `debian:latest`).
Conteneur	Instance exécutable d'une image (ex: un serveur Nginx).
Dockerfile	Fichier de configuration pour construire une image.
Volume	Stockage persistant pour les données (ex: base de données).
Docker Hub	Registre public d'images (comme GitHub pour les conteneurs).
Docker Compose	Outil pour orchestrer plusieurs conteneurs via un fichier YAML.

Jobs Réalisés

Job 01 : Installation de Docker

Objectif : Installer Docker CLI sur une VM Debian (8 Go DD, 1 Go RAM, 1 vCPU).

Commandes :

```
# Mise à jour du système
sudo apt update && sudo apt upgrade -y

# Installation des dépendances
sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release

# Ajout du dépôt Docker
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/debian $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null

# Installation de Docker
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io

# Vérification
sudo docker run my-hello-world
```

```
root@debian-54b3e820:~# docker run my-helloworld
Hello from my custom Debian container!
```

Job 02 : Test avec "hello-world"

Objectif : Valider l'installation avec le conteneur `hello-world`.

Commandes :

```
docker run hello-world
docker ps -a  # Voir les conteneurs
docker images # Voir les images
```

Résultat :

Hello from Docker!
This message shows that your installation appears to be working correctly.

Job 03 : Dockerfile pour "hello-world"

Objectif : Recréer le conteneur `hello-world` depuis une image Debian minimale.

Dockerfile :

```
FROM debian:latest
RUN apt update && apt install -y figlet
CMD ["figlet", "Hello Docker!"]
```

Commandes :

```
docker build -t hello-docker .
docker run hello-docker
```

Job 04 : Conteneur SSH

Objectif : Créer un conteneur SSH avec redirection de port (port 2222).

Dockerfile :

```
FROM debian:latest
RUN apt update && apt install -y openssh-server
RUN echo 'root:root123' | chpasswd
RUN mkdir /var/run/sshd
EXPOSE 22
CMD ["/usr/sbin/sshd", "-D"]
```

Commandes :

```
docker build -t ssh-container .
docker run -d -p 2222:22 ssh-container
ssh root@localhost -p 2222  # Mot de passe: root123
```

Job 05 : Alias Docker

Objectif : Simplifier les commandes Docker avec des alias.

Fichier `~/.bashrc` :

Commande :

```
source ~/.bashrc
```

```
laplateforme@debian-54b3e820: ~
# Note: PS1 is set in /etc/profile, and the default umask is defined
# in /etc/login.defs. You should not need this unless you want different
# defaults for root.
# PS1='${debian_chroot:+($debian_chroot)}\h:\w\$ '
# umask 022

# You may uncomment the following lines if you want `ls' to be colorized:
 export LS_OPTIONS='--color=auto'
 eval "$(dircolors)"
 alias ls='ls $LS_OPTIONS'
 alias ll='ls $LS_OPTIONS -l'
 alias l='ls $LS_OPTIONS -lA'
 force_color_prompt=yes
#
# Some more alias to avoid making mistakes:
# alias rm='rm -i'
# alias cp='cp -i'
# alias mv='mv -i'



# Prompt colorisé
PS1='\[\033[01;31m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$ '

# Docker aliases
alias dps='docker ps'
alias dpsa='docker ps -a'
alias di='docker images'
alias drm='docker rm'
alias drmi='docker rmi'
alias dstop='docker stop'
alias dstart='docker start'
alias drun='docker run'
alias dlogs='docker logs'
alias dexec='docker exec -it'
alias dbuild='docker build -t'
alias dprune='docker system prune -f'
alias dstopall='docker stop $(docker ps -q)'
alias drmall='docker rm $(docker ps -aq)'
```

Job 06 : Volumes entre conteneurs

Objectif : Partager un volume entre deux conteneurs.

Commandes :

```
# Créer un volume
docker volume create shared_volume

# Lancer deux conteneurs avec le volume
docker run -d --name container1 -v shared_volume:/data alpine sleep infinity
docker run -it --name container2 -v shared_volume:/data alpine sh
```

Job 07 : Docker Compose (Nginx + FTP)

Objectif : Créer deux conteneurs (Nginx + FTP) avec un volume partagé.

Fichier `docker-compose.yml` :

```
version: '3'
services:
  nginx:
    image: nginx:latest
    ports:
```

```
    - "8080:80"
  volumes:
    - web_data:/usr/share/nginx/html
ftp:
  image: stilliard/pure-ftpd
  ports:
    - "21:21"
    - "30000-30009:30000-30009"
  volumes:
    - web_data:/home/ftpuser
  environment:
    - FTP_USER_NAME=ftpuser
    - FTP_USER_PASS=ftppass
    - FTP_USER_HOME=/home/ftpuser
volumes:
  web_data:
```

Commandes :

```
docker-compose up -d
echo "<h1>Prénom NOM</h1>" > index.html
ftp localhost  # Envoyer le fichier index.html
```

```
laplateforme@debian-54b3e820: ~
  GNU nano 8.4                    docker-compose.yml *
services:
  nginx:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - web-data:/usr/share/nginx/html

  ftp:
    image: garethflowers/ftp-server
    environment:
      - FTP_USER=user
      - FTP_PASS=password
    ports:
      - "21:21"
      - "40000-40009:40000-40009"
    volumes:
      - web-data:/home/user

volumes:
  web-data:

^G Aide      ^O Écrire     ^F Chercher   ^K Couper     ^T Exécuter
^X Quitter   ^R Lire fich. ^\ Remplacer  ^U Coller     ^J Justifier
```

Job 08 : Conteneur Nginx personnalisé

Objectif : Créer un conteneur Nginx sans utiliser une image existante.

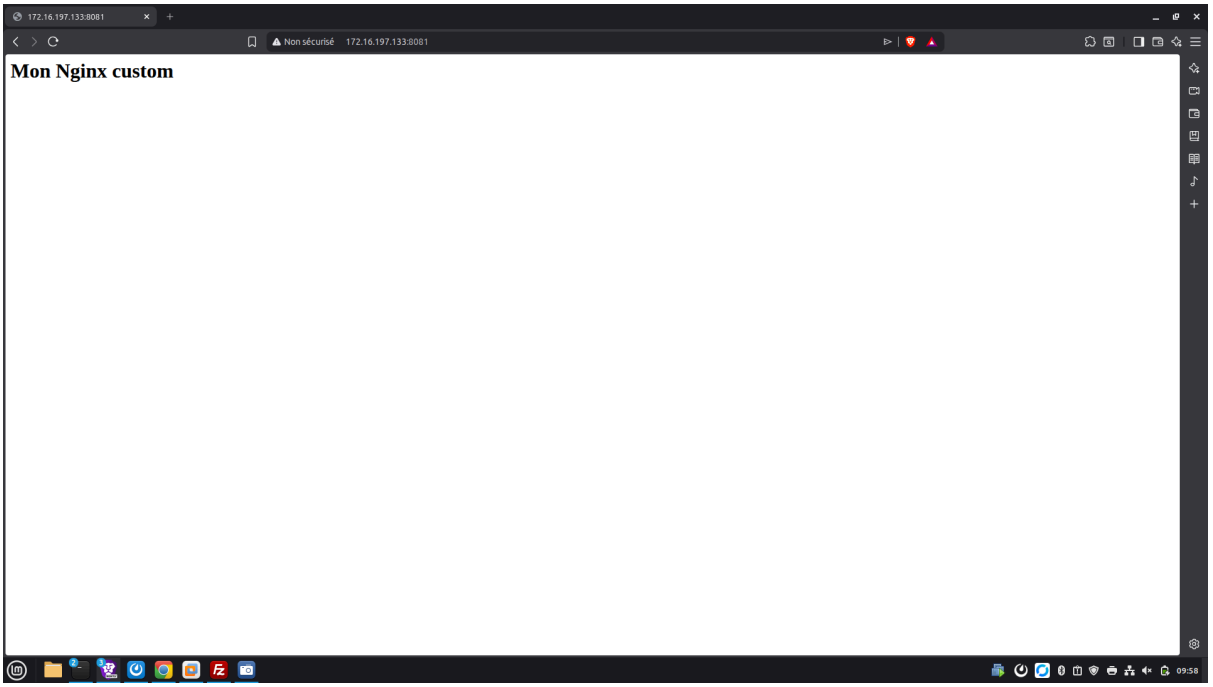
Dockerfile :

```
FROM debian:latest
RUN apt update && apt install -y nginx
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Commandes :

docker build -t custom-nginx .
docker run -d -p 8080:80 custom-nginx

```
root@debian-54b3e820:~/job08# docker run -d -p 8081:80 --name custom-ngi
nx my-nginx
05120ae784283ed91c9ffbd6b61226216292155e58dbaf29ec92785feefac944
```



Job 09 : Registry local avec UI

Objectif : Créer un registry local avec une interface web.

Commandes :

Lancer le registry
docker run -d -p 5000:5000 --name registry registry:2

Lancer l'UI
docker run -d -p 8081:80 --name registry-ui -e REGISTRY_URL=http://registry:5000 joxit/docker-registry-ui:static

---

Job 10 : Scripts d'automatisation

Objectif : Automatiser l'installation et la désinstallation de Docker.

Script `install_docker.sh` :

```
#!/bin/bash
sudo apt update && sudo apt upgrade -y
sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/debian $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io
```

Script `uninstall_docker.sh` :

```
#!/bin/bash
sudo apt purge -y docker-ce docker-ce-cli containerd.io
sudo rm -rf /var/lib/docker
sudo rm -rf /var/lib/containerd
```

```
root@debian-54b3e820:~# mkdir ~/job10 && cd ~/job10
root@debian-54b3e820:~/job10# nano docker_clean.sh
root@debian-54b3e820:~/job10# nano docker_install.sh
root@debian-54b3e820:~/job10# chmod +x docker_clean.sh
root@debian-54b3e820:~/job10# chmod +x docker_install.sh
```

Job 11 : Portainer et alternatives

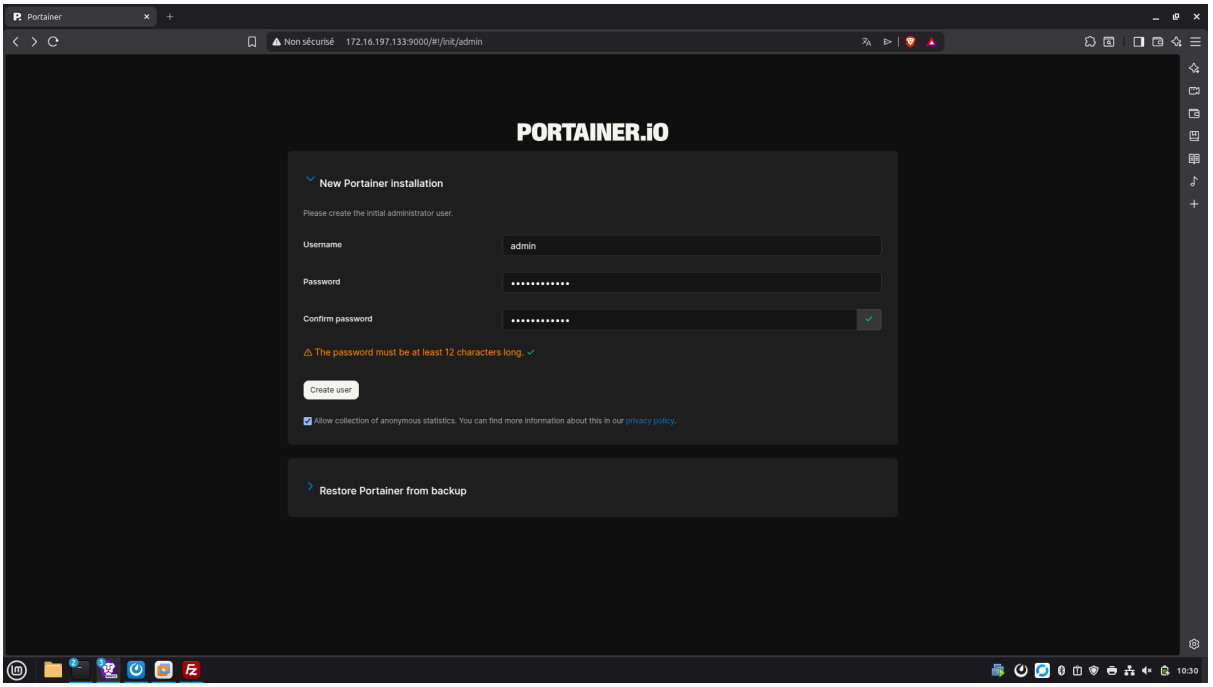
Objectif : Utiliser Portainer pour gérer Docker via une interface graphique.

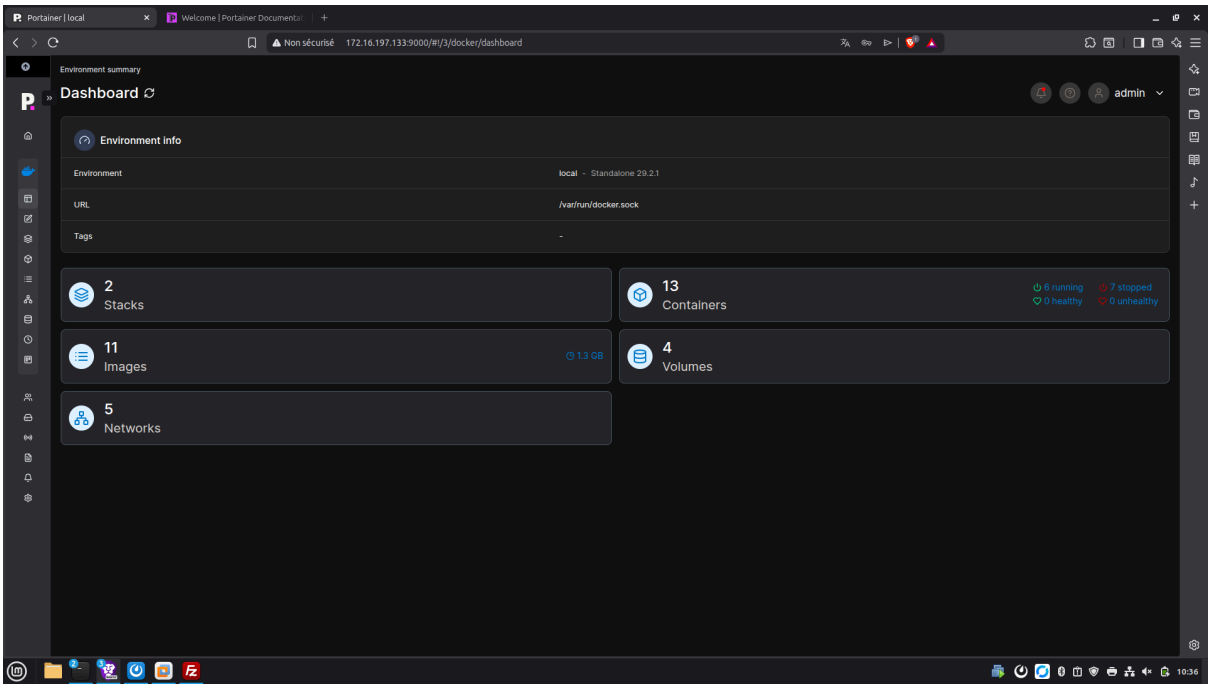
Commandes :

```
docker run -d -p 9000:9000 --name portainer --restart always -v
/var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer
```

Alternatives :

- **Rancher** : Gestion avancée de clusters.
- **Kubernetes Dashboard** : Pour les environnements Kubernetes.





Pour Aller Plus Loin : XAMPP-like

Objectif : Créer un environnement complet avec Nginx, PHP, MariaDB et FTP.

Fichier `docker-compose.yml` :

```yaml
version: '3'
services:
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./web:/usr/share/nginx/html
  php:
    image: php:fpm
    volumes:
      - ./web:/usr/share/nginx/html
  mariadb:
    image: mariadb:latest
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: mydb
      MYSQL_USER: user
      MYSQL_PASSWORD: userpass
    volumes:
      - db_data:/var/lib/mysql
  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    ports:
      - "8080:80"
    environment:
      PMA_HOST: mariadb
  ftp:
    image: stilliard/pure-ftpd
    ports:
      - "21:21"
      - "30000-30009:30000-30009"
    volumes:
      - ./web:/home/ftpuser
    environment:
      - FTP_USER_NAME=ftpuser
      - FTP_USER_PASS=ftppass
      - FTP_USER_HOME=/home/ftpuser
volumes:
  db_data:
```

laplateforme@debian-54b3e820: ~

```
root@debian-54b3e820:~# mkdir ~/xampp && cd ~/xampp
root@debian-54b3e820:~/xampp# mkdir nginx-php
root@debian-54b3e820:~/xampp# nano nginx-php/nginx.conf
root@debian-54b3e820:~/xampp# nano nginx-php/Dockerfile
root@debian-54b3e820:~/xampp# nano docker-compose.yml
root@debian-54b3e820:~/xampp# docker compose up -d
[+] up 32/32
 ✔ Image phpmyadmin:latest Pulled                                103.5s
 ✔ Image mariadb:latest    Pulled                                67.9ss
[+] Building 94.5s (10/10) FINISHED
 => [internal] load local bake definitions                                                                                  0.0s
 => => reading from stdin 478B                                                                                              0.0s
 => [internal] load build definition from Dockerfile                                                                        0.1s
 => => transferring dockerfile: 385B                                                                                        0.0s
 => [internal] load metadata for docker.io/library/debian:stable-slim                                                       1.6s
 => [internal] load .dockerignore                                                                                           0.1s
 => => transferring context: 2B                                                                                             0.0s
 => CACHED [1/3] FROM docker.io/library/debian:stable-slim@sha256:85dfcffff3c1e193877f143d05eaba8ae7f3f95cb0a32e0bc04a448077e1ac69  0.3s
 => => resolve docker.io/library/debian:stable-slim@sha256:85dfcffff3c1e193877f143d05eaba8ae7f3f95cb0a32e0bc04a448077e1ac69  0.2s
 => [internal] load build context                                                                                           0.1s
 => => transferring context: 366B                                                                                           0.0s
 => [2/3] RUN apt-get update && apt-get install -y     nginx php-fpm php-mysql &&     rm -rf /var/lib/apt/lists/* &&     mkdir -p /var/www  76.8s
 => [3/3] COPY nginx.conf /etc/nginx/sites-available/default                                                                0.2s
 => exporting to image                                                                                                     14.0s
 => => exporting layers                                                                                                    10.4s
 => => exporting manifest sha256:441d9eea0983d857cc41df2bbfa5b8ac680b984f25d5eed09737687d65622670                           0.0s
 => => exporting config sha256:3c931ba38a1c7044d78807998c2e262d38c1799542301ae5530f158acdd30d19                             0.0s
 => => exporting attestation manifest sha256:91455c3f176483c223503f93e0e18319b77bebe747b4c5ae6aa0e7bb807c3455               0.0s
 => => exporting manifest list sha256:ca27c31188dac5e890df67c0bcd53c0a6e4d95e82576e85bab80639b6204ac8c                      0.0s
[+] up 40/40g to docker.io/library/xampp-web:latest                                                                          0.0s
 ✔ Image phpmyadmin:latest      Pulled                                                                                     103.5s
 ✔ Image mariadb:latest         Pulled                                                                                      67.9s
 ✔ Image xampp-web              Built                                                                                       94.8s
 ✔ Network xampp_default        Created                                                                                     0.2s
 ✔ Volume xampp_db-data         Created                                                                                     0.0s
 ✔ Volume xampp_web-data        Created                                                                                     0.0s
 ✔ Container xampp-db-1         Started                                                                                     3.1s
 ✔ Container xampp-ftp-1        Started                                                                                     3.2s
 ✔ Container xampp-phpmyadmin-1 Started                                                                                     4.9s
```

08:32